

Clustering Analysis and KMeans

Nimisha Roy

Lecturer, SCI, College of Computing, Georgia Tech

60+ hours on 16 GPU nvidia CUDA cluster.



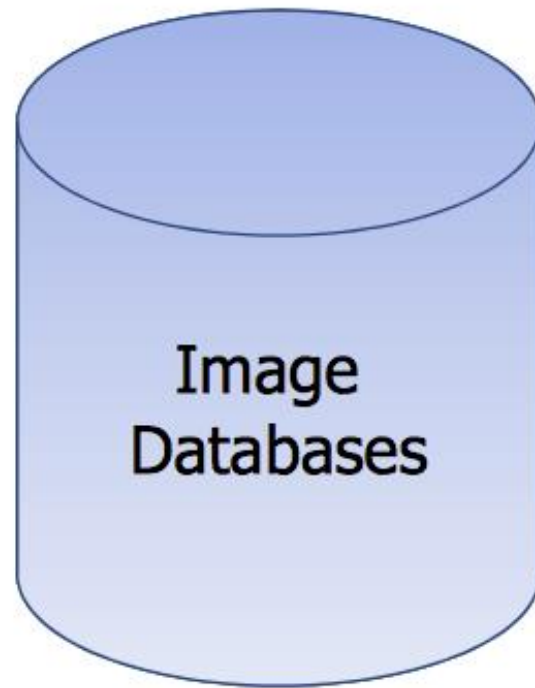
Outline

- Clustering
- Distance functions
- K-Means algorithm
- Analysis of K-Means

Outline

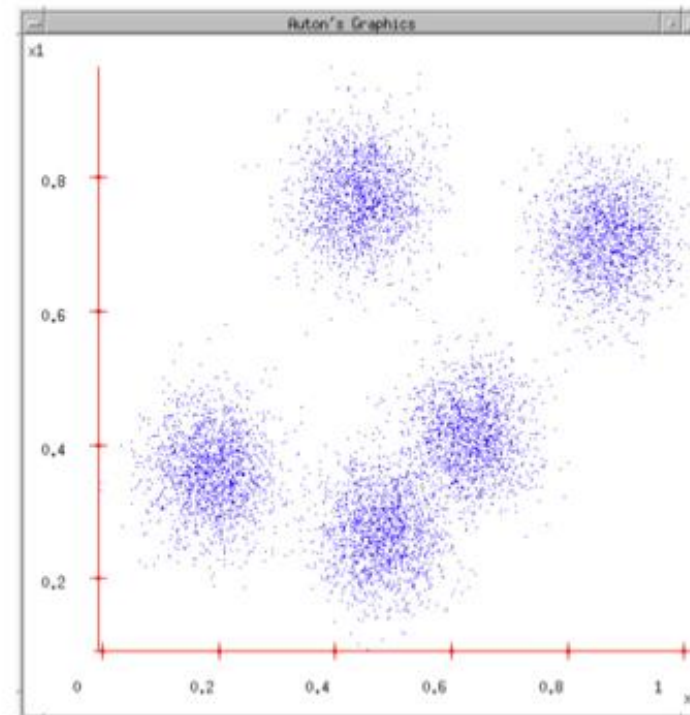
- **Clustering**
 - Distance functions
 - K-Means algorithm
 - Analysis of K-Means

Clustering Images



Goal of clustering:

Divide object into groups,
and objects within a group
are more similar than
those outside the group



Clustering Other Objects: Similarity Metric Matters

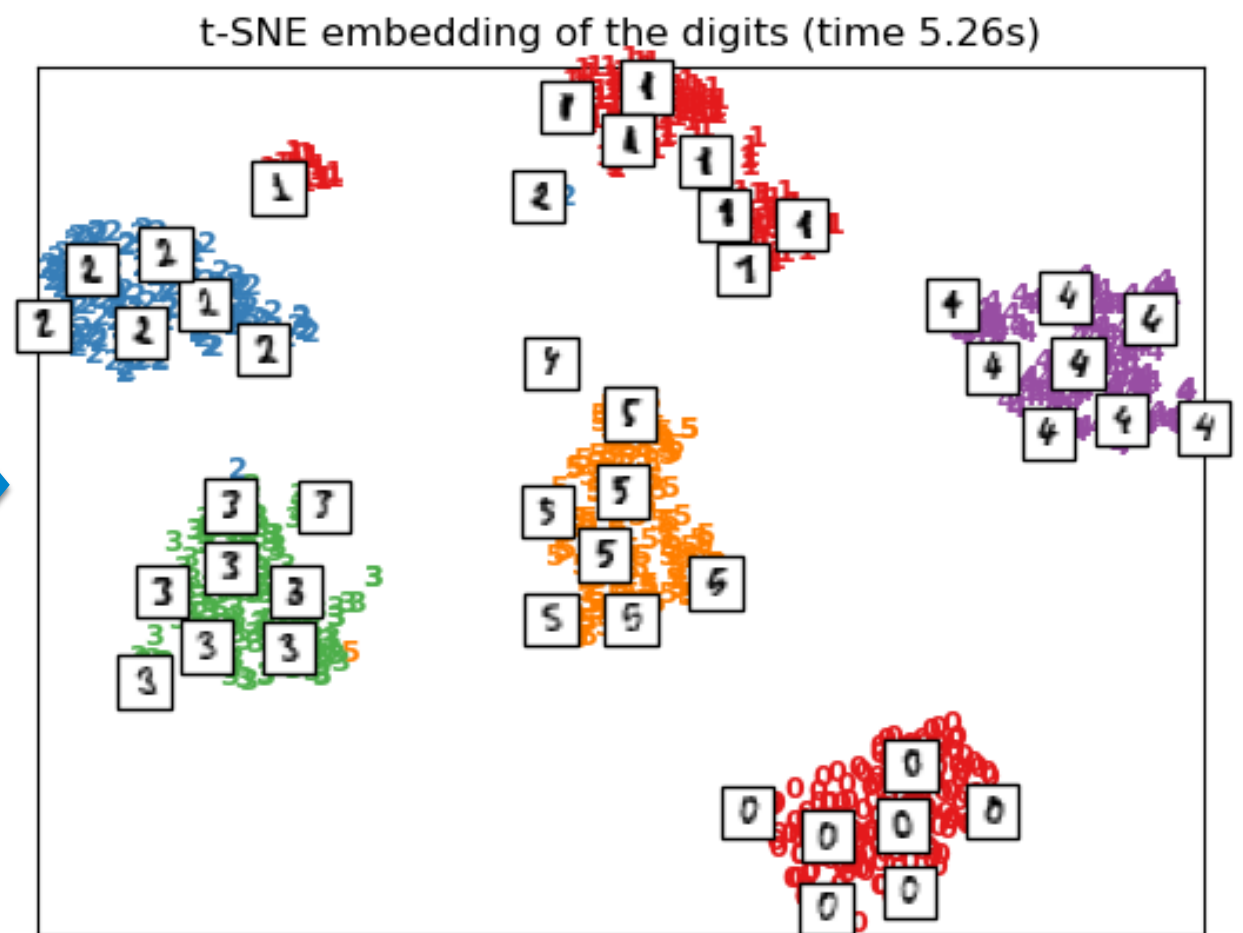
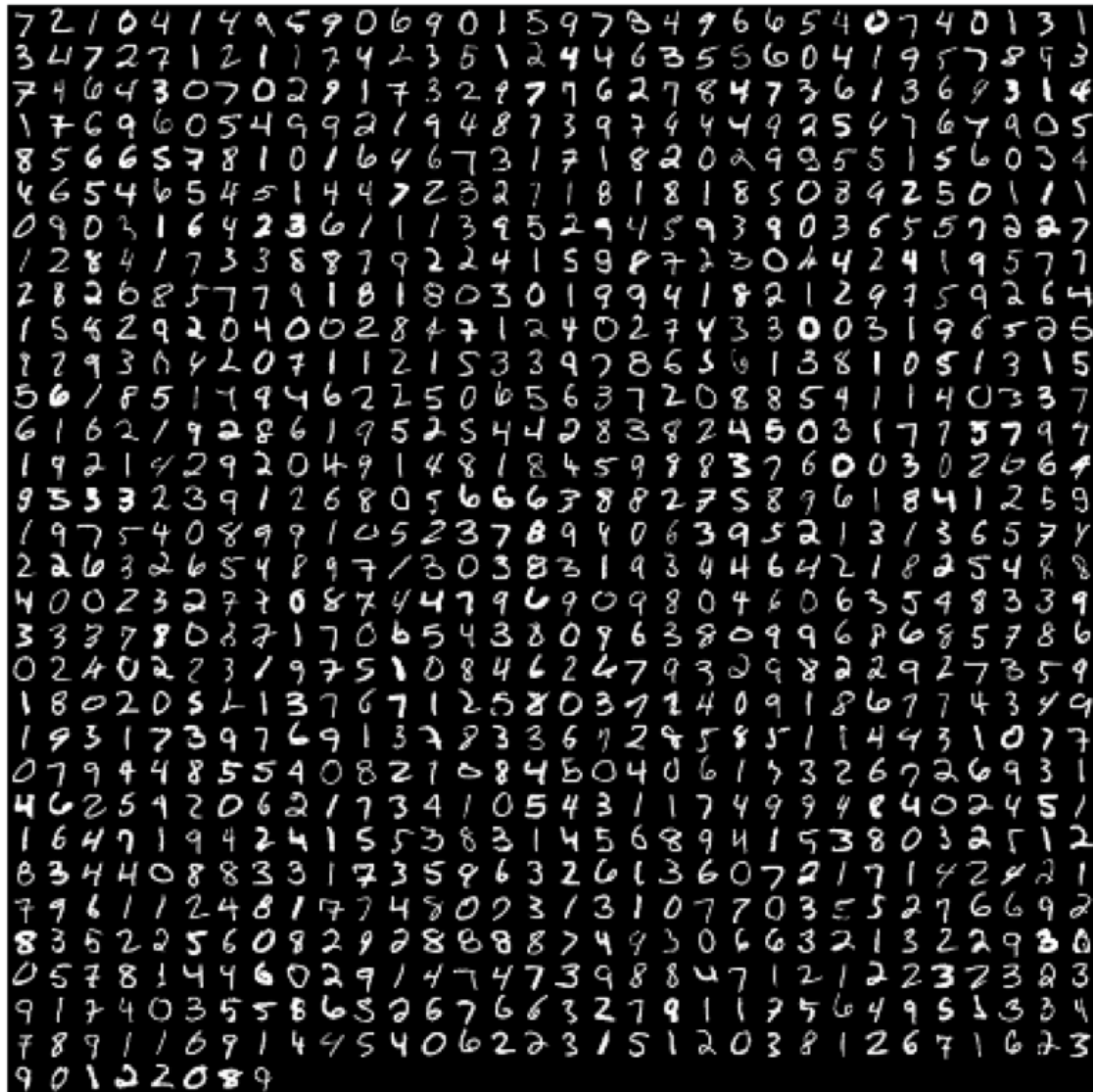


Belarusian *Piotr*
Azerbaijani *Pyotr*
Greek *Petros*
Italian *Pietro*
Portuguese *Pedro*
French *Pierre*
Italian *Piero*
Dutch *Peter*
Danish *Peder*
Couldn't find it – Finish? *Peka*
Irish *Peadar*

Linguistic Similarity

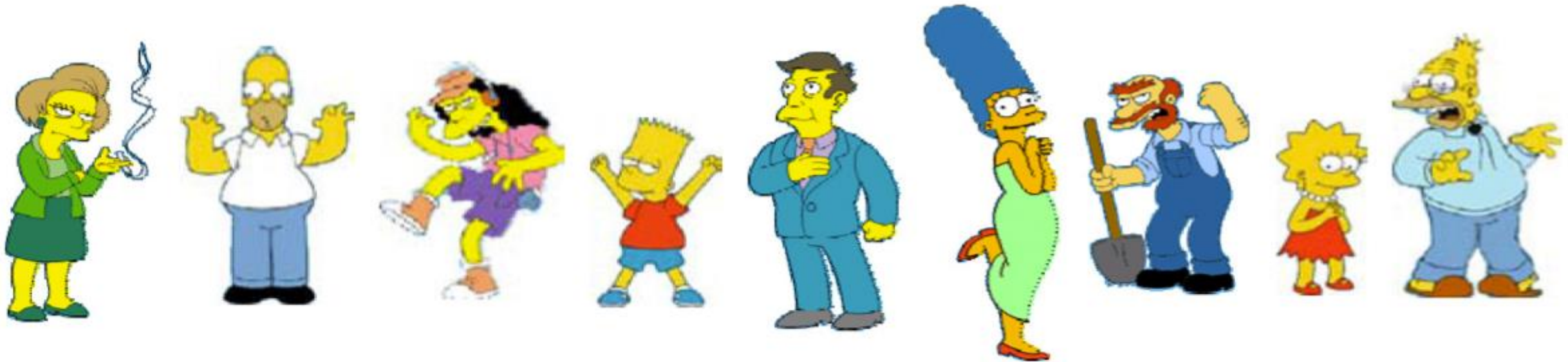


Clustering hand digits



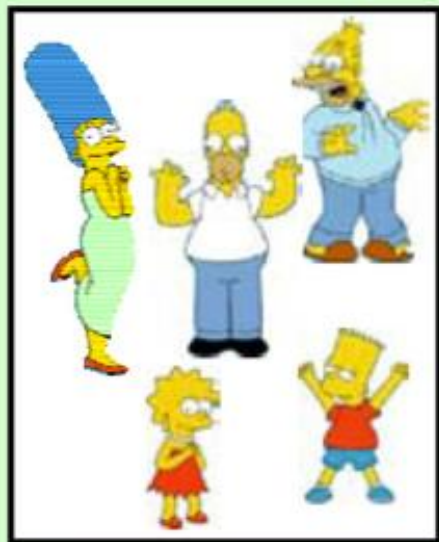
[Image credit: Scikit learn](#)

Clustering is Subjective



What is consider similar/dissimilar?

Clustering is subjective



Simpson's Family



School Employees



Females



Males

So, What is Clustering in General?

- You pick your similarity/dissimilarity function
- The algorithm figures out the grouping of objects based on the chosen similarity/dissimilarity function
 - Points within a cluster is similar
 - Points across clusters are not so similar
- Issues for clustering
 - How to represent objects? (Vector space? Normalization?)
 - What is a similarity/dissimilarity function for your data?
 - What are the algorithm steps?

How is clustering useful?

- Identifying communities or categories of items to better understand our data
 - Are there unexpected clusters?
 - Are there more clusters than classes in a labeled dataset?
 - Are there data points that don't fall in any groups?
- Streamlining tasks such as database search and data labeling
- Identifying whether there are natural separations in our dataset before proceeding to a classification task
- Helping us make decisions about the quality of our data
- *Many other use cases...*

Outline

- Clustering
- **Distance functions**
- K-Means algorithm
- Analysis of K-Means

Properties of distance functions

- Desired properties of distance functions:
- **Symmetry:** $d(\mathbf{x}, \mathbf{y}) = d(\mathbf{y}, \mathbf{x})$
 - Otherwise you could claim “Alex looks like Taylor, but Taylor looks nothing like Alex”
- **Positive separability:** $d(\mathbf{x}, \mathbf{y}) = 0$, if and only if $\mathbf{x} = \mathbf{y}$
 - Otherwise there are objects that are different, but you cannot tell them apart
- **Triangular inequality:** $d(\mathbf{x}, \mathbf{y}) \leq d(\mathbf{x}, \mathbf{z}) + d(\mathbf{z}, \mathbf{y})$
 - Otherwise you could claim “Alex looks like Taylor, and Alex looks like Chris, but Taylor does not look like Chris”

Distance functions for vectors

- Suppose two data points, both in $\mathbb{R}^D$

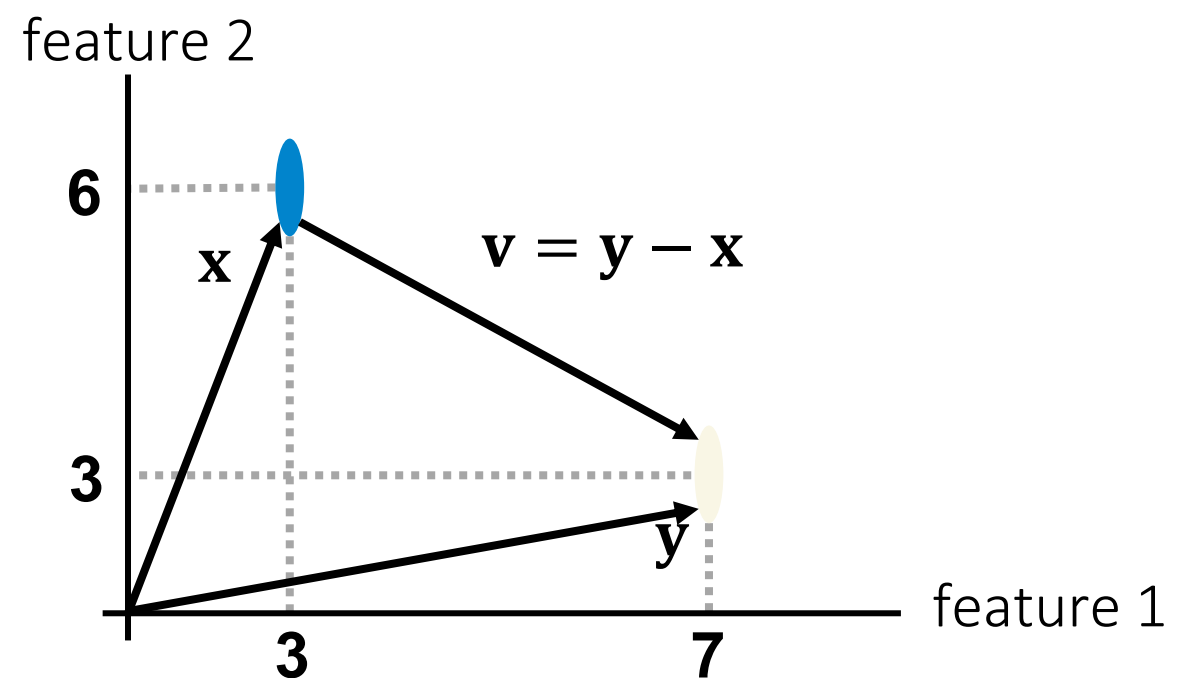
$$\mathbf{x} = (x_1, x_2, \dots, x_D)$$

$$\mathbf{y} = (y_1, y_2, \dots, y_D)$$

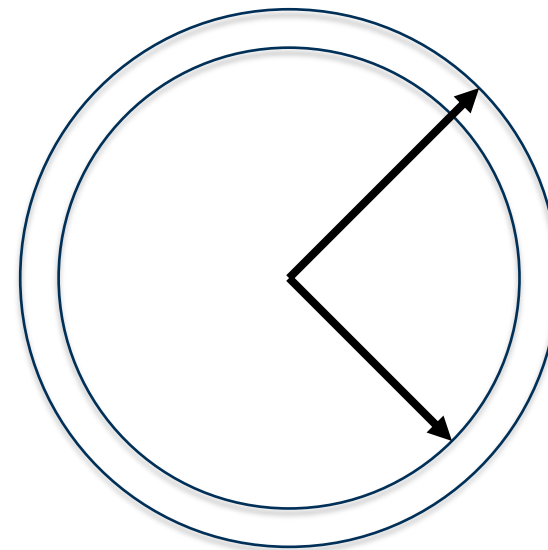
- Euclidean distance (L2-norm): $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^D (x_i - y_i)^2}$
- Minkowski distance (Lp-norm): $d(\mathbf{x}, \mathbf{y}) = \sqrt[p]{\sum_{i=1}^D (x_i - y_i)^p}$
 - Euclidean distance: $p = 2$
 - Manhattan distance: $p = 1, d(\mathbf{x}, \mathbf{y}) = \sum_{i=1}^D |x_i - y_i|$
 - “Inf”-distance: $p = \infty, d(\mathbf{x}, \mathbf{y}) = \max_i |x_i - y_i|$

Example

- Euclidean distance: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^D (x_i - y_i)^2} = \sqrt{(7 - 3)^2 + (3 - 6)^2} = 5$
- Manhattan distance: $d(\mathbf{x}, \mathbf{y}) = |7 - 3| + |3 - 6| = 7$
- “Inf”-distance: $d(\mathbf{x}, \mathbf{y}) = \max(|7 - 3|, |3 - 6|) = 4$



Some problems with Euclidean distance



Curse of Dimensionality

- Refers to the phenomenon where the efficiency and effectiveness of algorithms deteriorate as the dimensionality of the data increases exponentially.
- In high-dimensional spaces, data points become sparse, making it challenging to discern meaningful patterns or relationships due to the vast amount of data required to adequately sample the space.
- Curse of Dimensionality significantly impacts ML algorithms in various ways. It leads to increased computational complexity, longer training times, and higher resource requirements. Moreover, it escalates the risk of overfitting and spurious correlations, hindering the algorithms' ability to generalize well to unseen data.

Hamming distance

- Manhattan distance is also called **Hamming distance** when all features are binary
 - Count the number of difference between two binary vectors
 - Example, $\mathbf{x}, \mathbf{y} \in \{0,1\}^{17}$

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
x	0	1	1	0	0	1	0	0	1	0	0	1	1	1	0	0	1
y	0	1	1	1	0	0	0	0	1	1	1	1	1	1	0	1	1

$$d(\mathbf{x}, \mathbf{y}) = 5$$

- When is the Hamming distance used?
 - When features are categorical
 - e.g., DNA sequences, yes/no survey answers, on/off flags.

Edit distance

- Transform one of the objects into the other, and measure how much effort it takes

x	I	N	T	E	*	N	T	I	O	N
y	*	E	X	E	C	U	T	I	O	N
	d	s	s		i	s				

- d: deletion (cost 5)
- s: substitution (cost 1)
- i: insertion (cost 2)

(these costs are arbitrary)

$$d(\mathbf{x}, \mathbf{y}) = 5 \times 1 + 3 \times 1 + 1 \times 2 = 10$$

Outline

- Clustering
- Distance functions
- **K-Means algorithm**
- Analysis of K-Means

Motivating example: Image compression



Source: [Reddit](#)

Communication bottlenecks



Source: [Giphy](#)



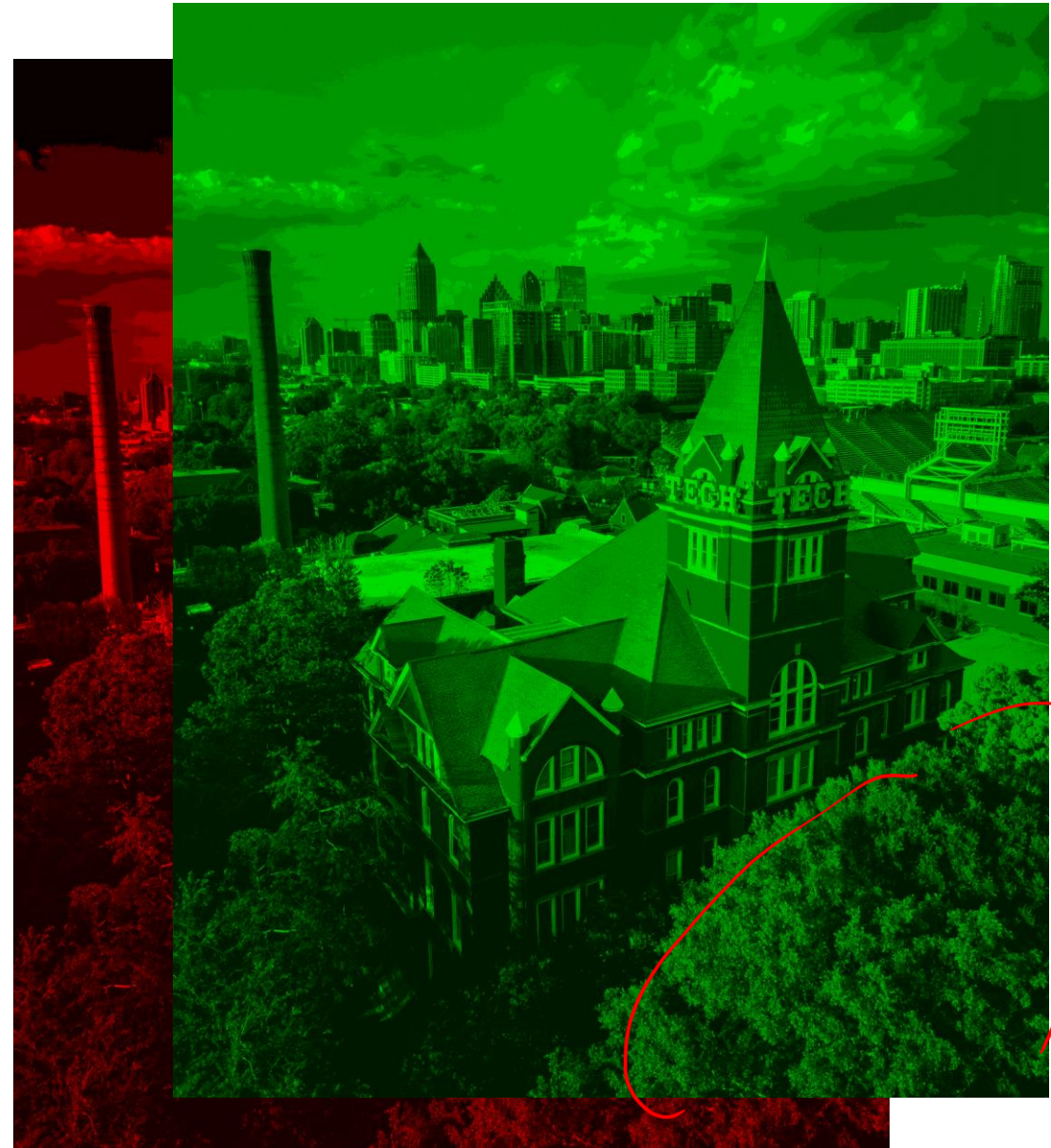
How is image data stored?



How is image data stored?



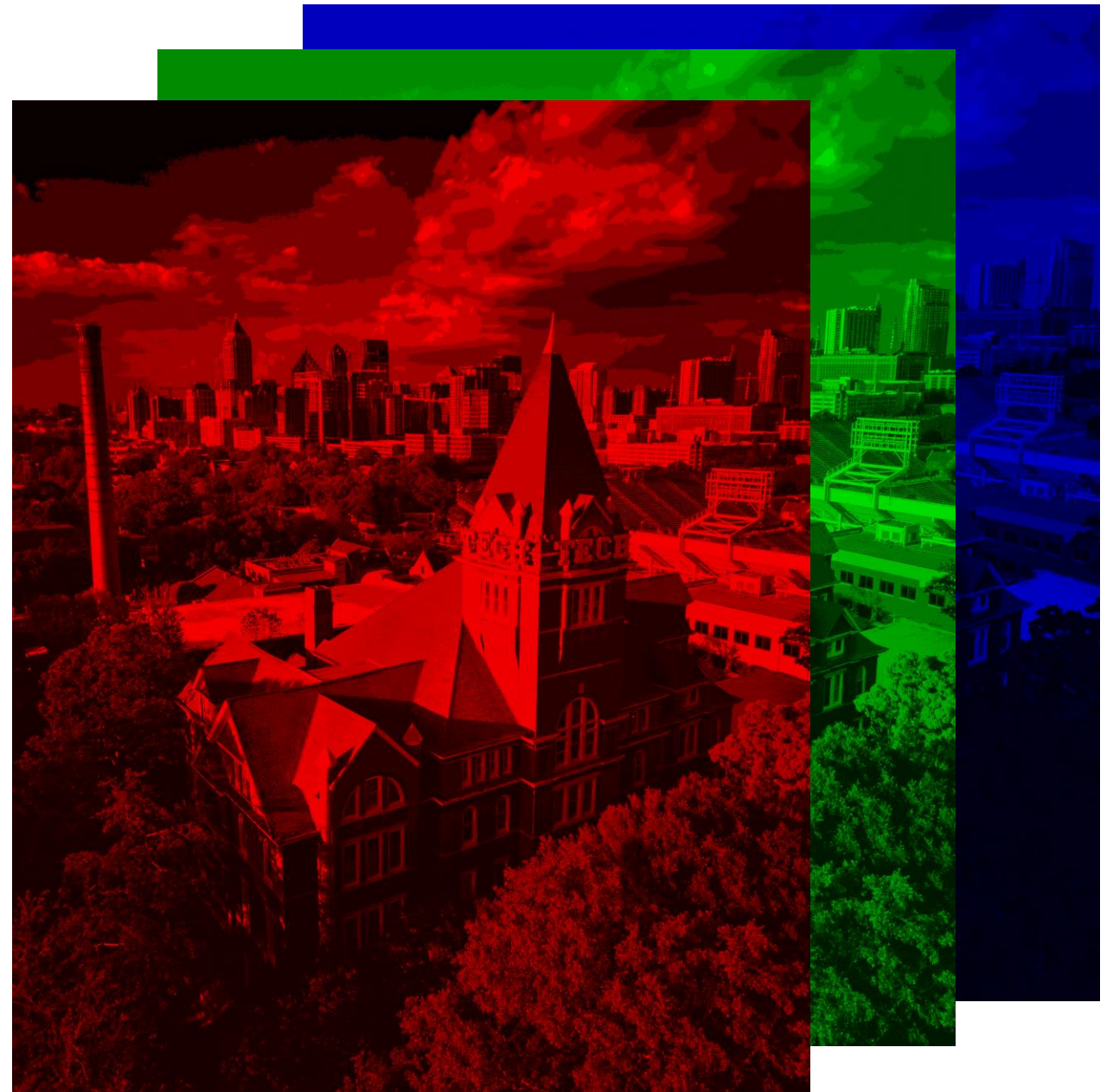
How is image data stored?



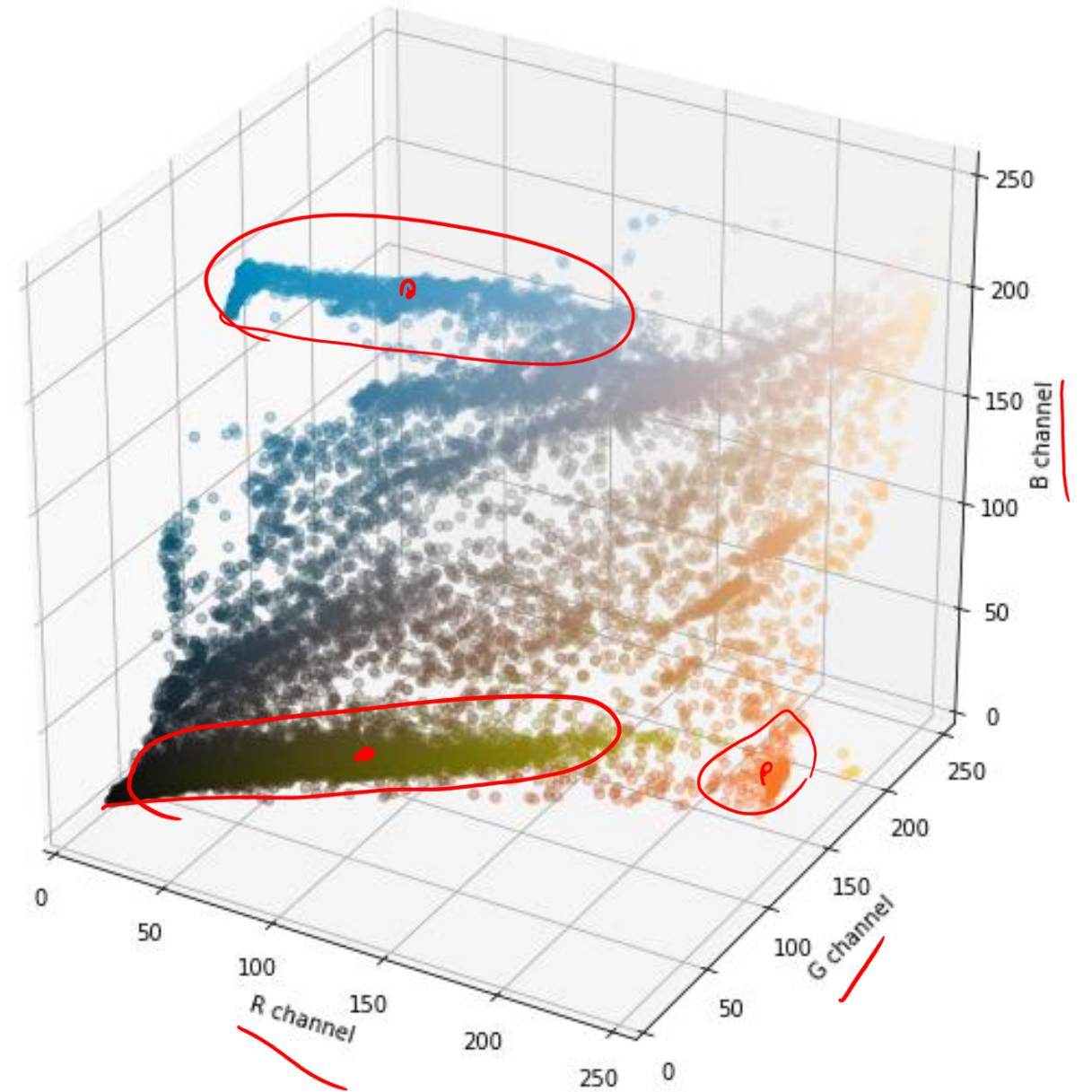
How is image data stored?



How is image data stored?



How are colors distributed in RGB space?



What if we encoded this image?



R	G	B
0	100	250
100	100	100
10	250	10
250	50	0
255	255	255
0	0	0



1	1	...	...	2
1	1	...	...	...
1	...	...	...	...
...	...	...	...	3
...	...	...	...	3
3	3	...	3	3

Compressed image



$K = 6$



$K = 12$



$K = 24$

Motivation 2: We can use compressed image to insert into an ML algorithm for training detecting sky or building or grass. It has much processed input than a complex rgb image input.

K-Means Algorithm

- Initialize k cluster centers, $\{c_1, c_2, \dots, c_k\}$, randomly
- Do
 - Decide the cluster memberships of each data point, x_i by assigning it to the nearest cluster center (**cluster assignment**)

$$\pi(i) = \underset{j=1, \dots, k}{\operatorname{argmin}} \quad \|x_i - c_j\|^2$$

- Adjust the cluster centers (**center adjustment**)

$$c_j = \frac{1}{|\{i: \pi(i) = j\}|} \sum_{i: \pi(i)} x_i$$

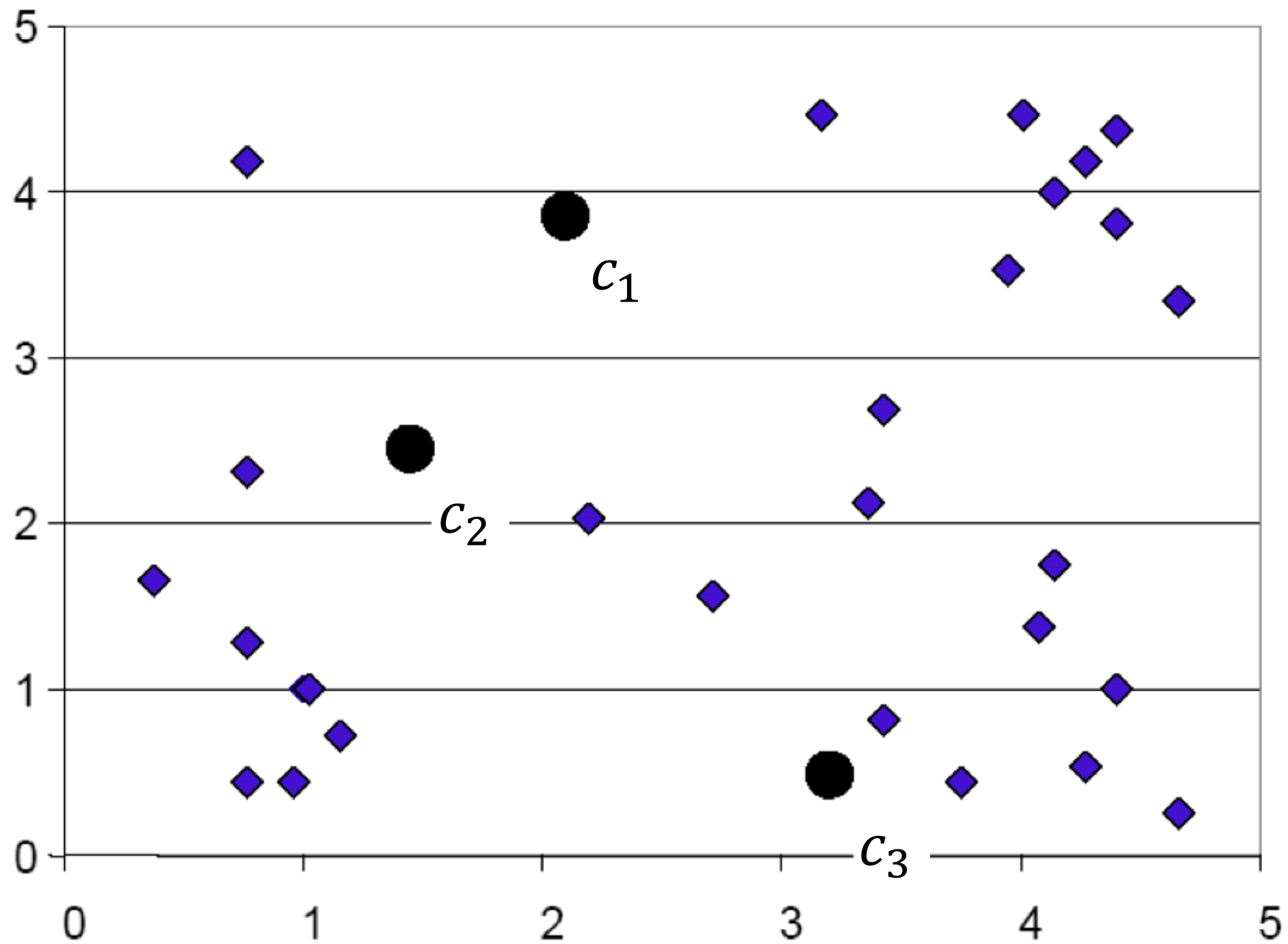
- While any cluster center has been changed

K-means algorithm (in plain English)

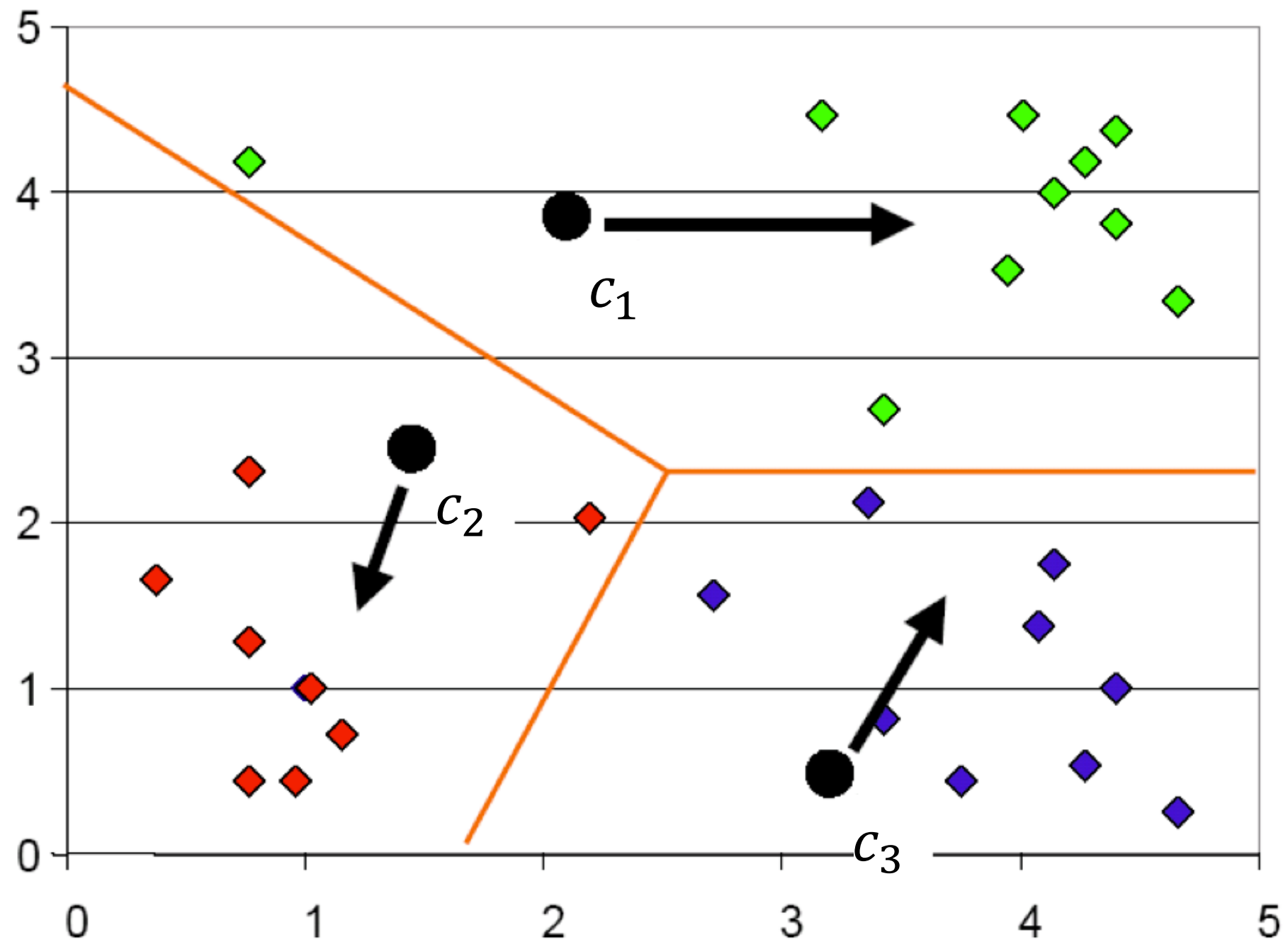
1. Initialize the number of clusters and their centers
2. Compute the distance between each point and each cluster center.
3. Assign each point the cluster id of the nearest cluster center
4. Recompute the cluster centers based on the cluster assignment to each point
5. Repeat steps 2 and 3 until convergence

Visualizing K Means

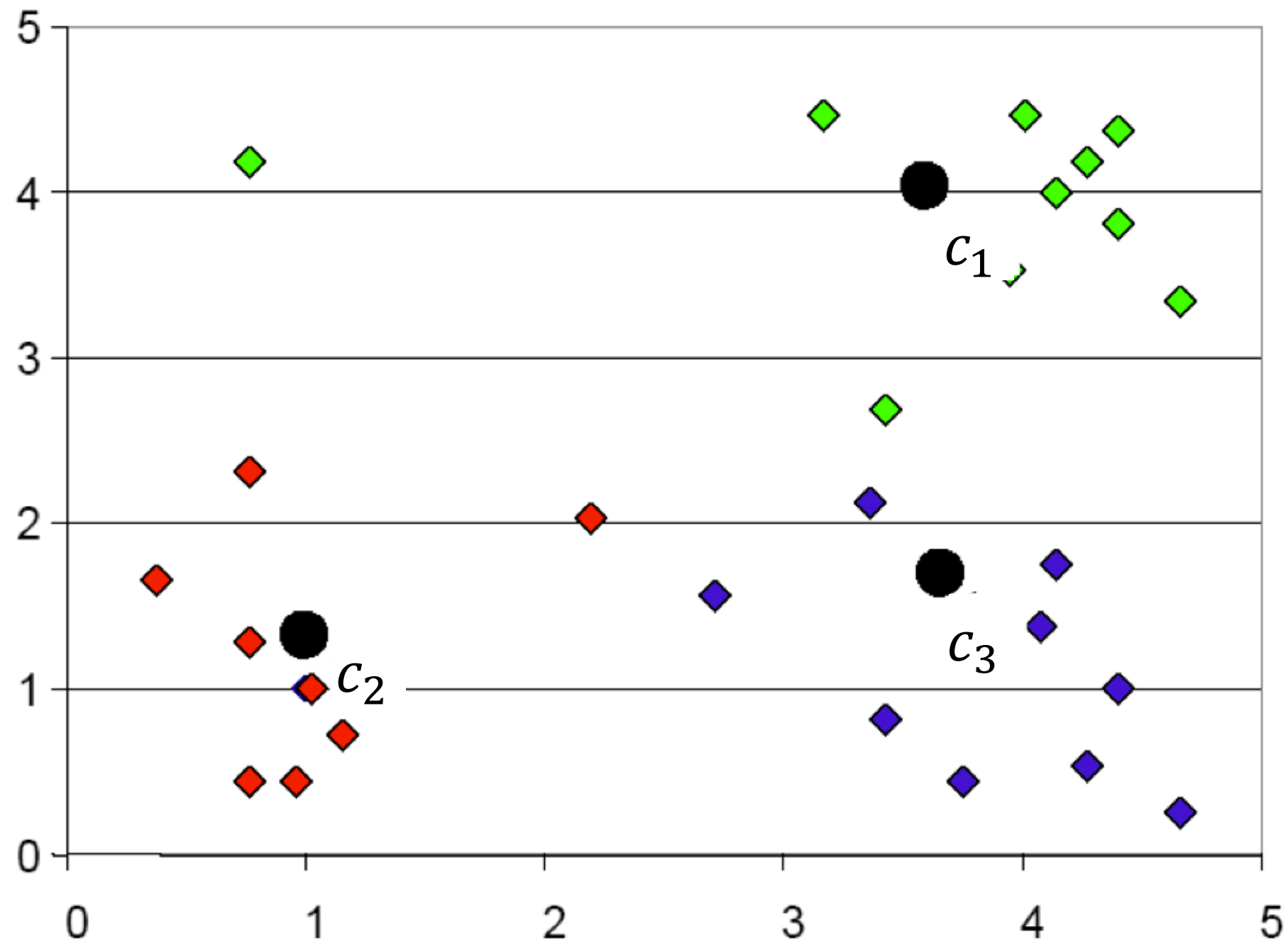
K-Means: Step 1



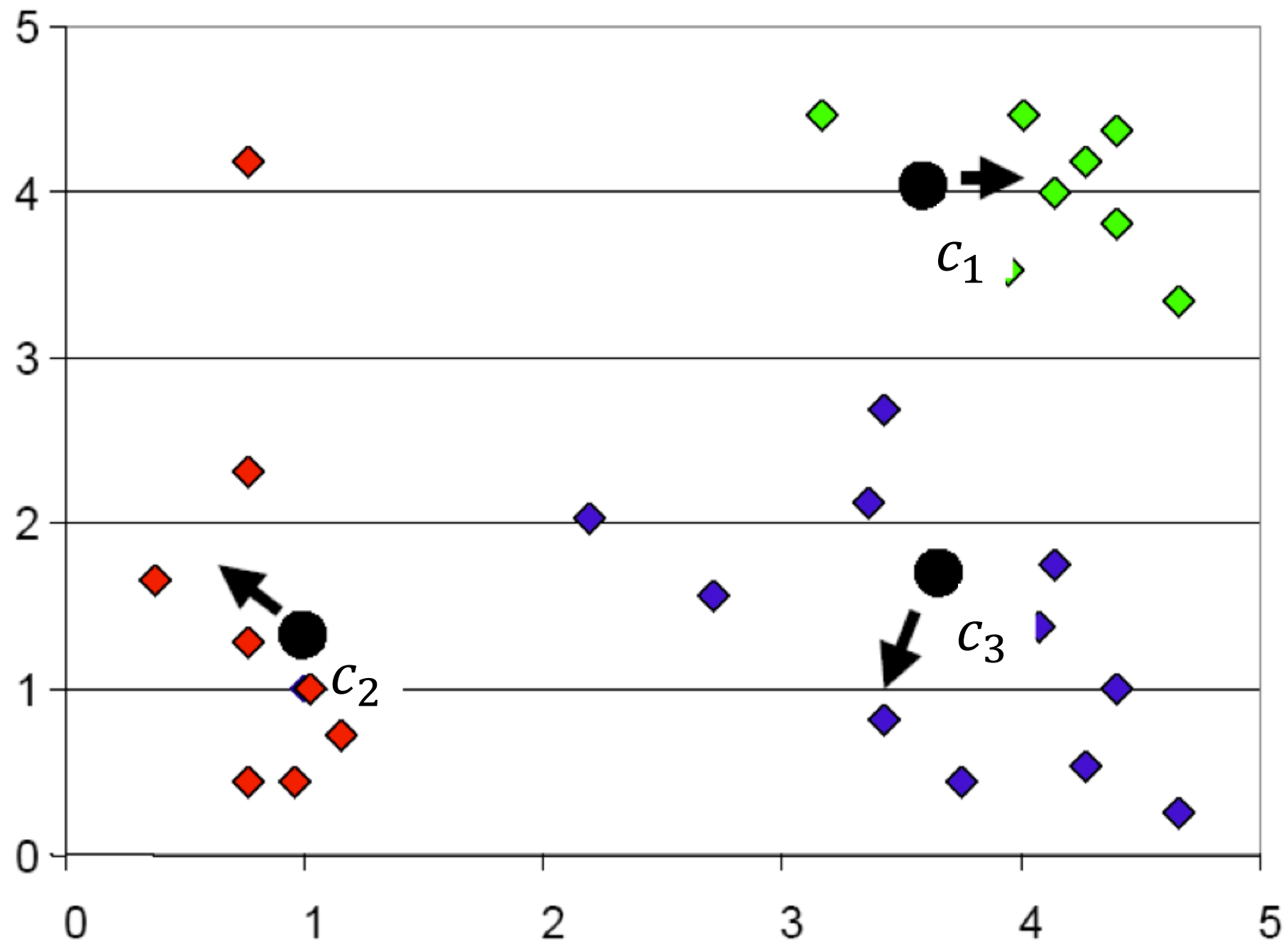
K-Means: Step 2



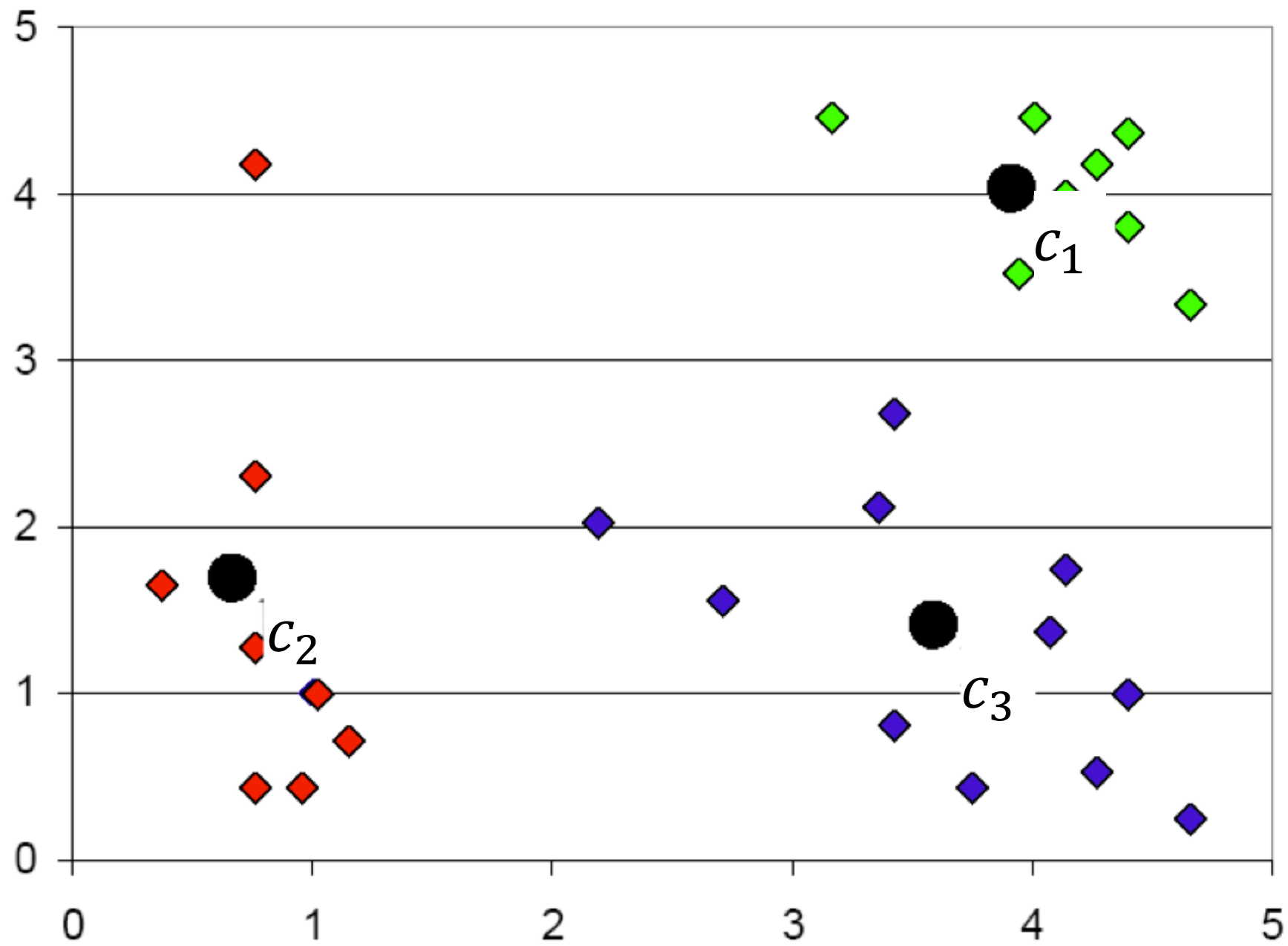
K-Means: Step 3



K-Means: Step 4



K-Means: Step 5



Some implementation questions

- Will different initializations lead to different results?
 - a. Always
 - b. Sometimes
 - c. Never

- Will the algorithm always stop after some iteration?
 - a. Yes
 - b. No (we have to set a maximum number of iterations)

Formal Statement of the Clustering Problem

- Given n data points, $\{x_1, x_2, \dots, x_n\} \ x \in R^d$
- Find k cluster centers, $\{c_1, c_2, \dots, c_k\} \ c \in R^d$
- And assign each datapoint i to one cluster, $\pi(i) \in \{1, \dots, k\}$
- Such that the averaged square distances from each datapoint to its respective cluster center is small

$$\min_{c, \pi} \sum_{i=1}^n \|x_i - c_{\pi(i)}\|^2$$

Outline

- Clustering
- Distance functions
- K-Means algorithm
- **Analysis of K-Means**

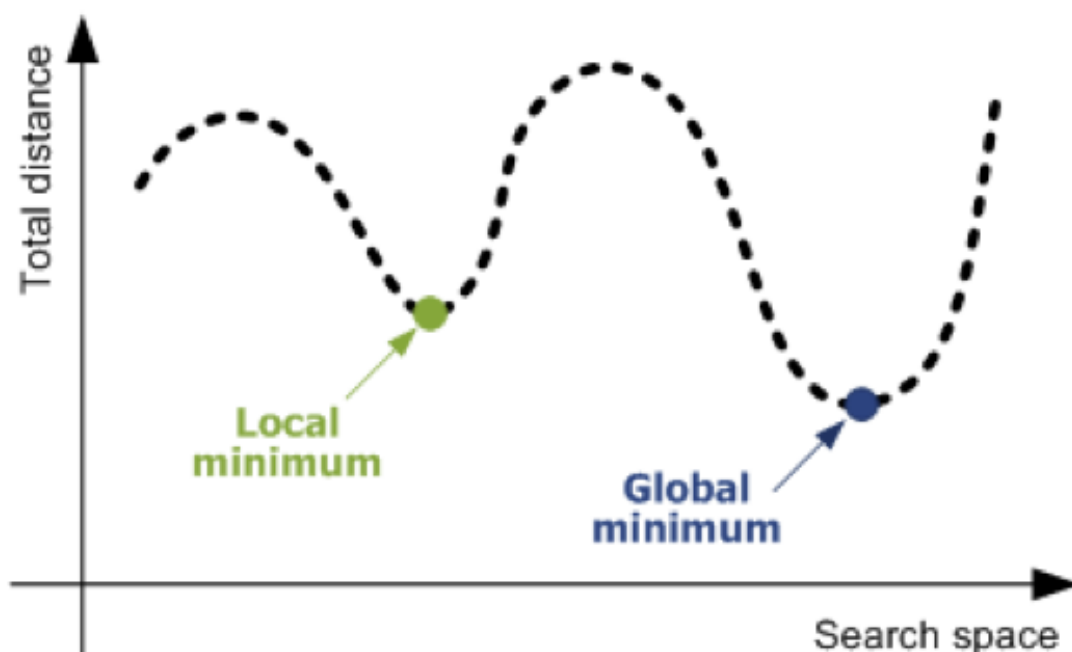
Clustering is NP-Hard

- Find k cluster centers, $\{c_1, c_2, \dots, c_k\} \subset \mathbb{R}^d$, and assign each data point i to one cluster, $\pi(i) \in \{1, \dots, k\}$, to minimize

$$\min_{c, \pi} \sum_{i=1}^n \|x_i - c_{\pi(i)}\|^2$$

NP-hard!

- A search problem over the space of discrete assignments
 - For all n data point together, there are k^n possibility
 - The cluster assignment determines cluster centers, and vice versa



- For all n data point together, there are k^n possibility

$X = \{A, B, C\}$

$n=3$ (data points)

$k=2$ clusters of two members

Cluster 1

Cluster 2

Convergence of K-Means

- Will kmeans objective oscillate?

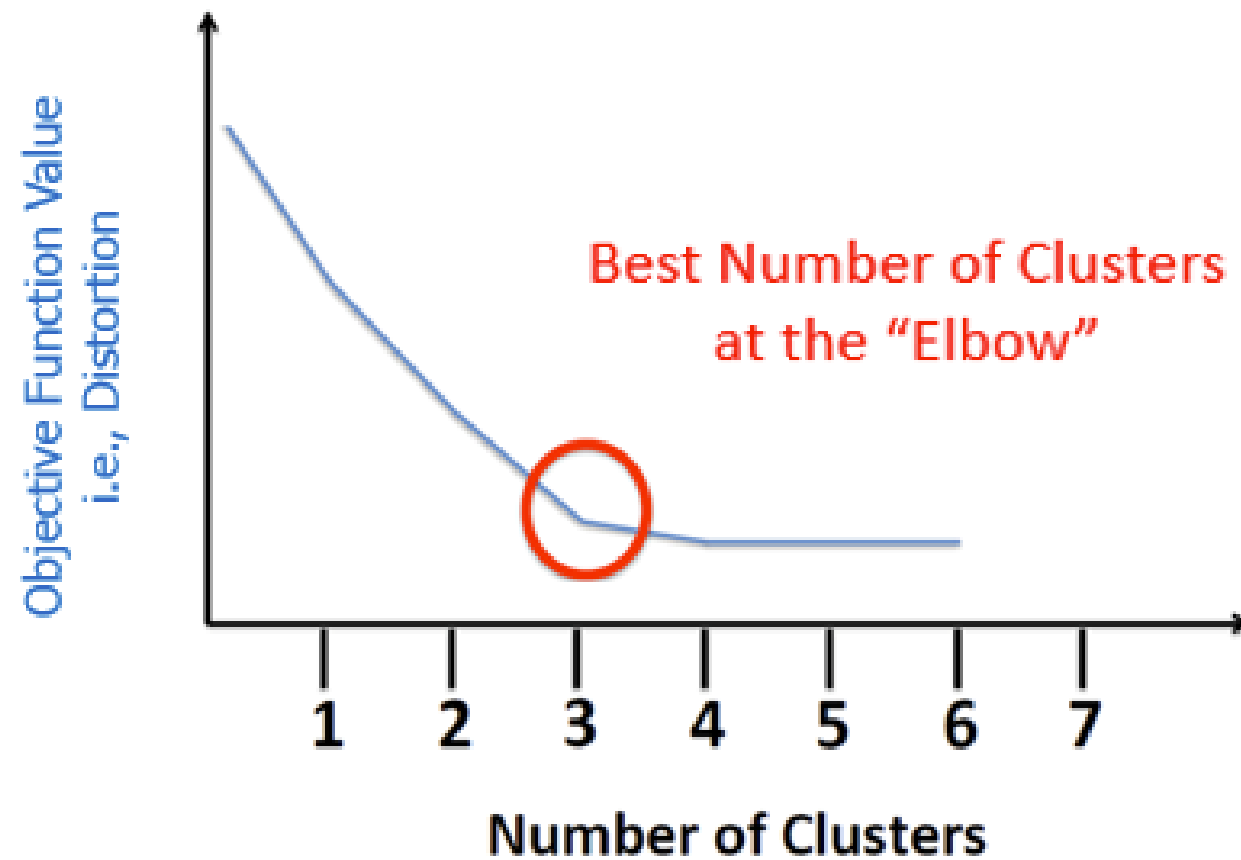
$$\min_{c, \pi} \sum_{i=1}^n \|x_i - c_{\pi(i)}\|^2$$

- The minimum value of the objective is finite
- Each iteration of kmeans algorithm decrease the objective
 - Cluster assignment step decreases objective
 - $\pi(i) = \operatorname{argmin}_{j=1, \dots, k} \|x_i - c_{\pi(j)}\|^2$ for each data point i
 - Center adjustment step decreases objective
 - $c_i = \frac{1}{|\{i: \pi(i)=j\}|} \sum_{i: \pi(i)=j} x_i = \operatorname{argmin}_c \sum_{i: \pi(i)=j} \|x_i - c_{\pi(j)}\|^2$

- Assume computing distance between two instances is $O(d)$ where d is the dimensionality of the vectors.
- Reassigning clusters for all datapoints:
 - $O(kn)$ distance computations (when there is one feature)
 - $O(knd)$ (when there is d features)
- Computing centroids: Each instance vector gets added once to some centroid (Finding centroid for each feature): $O(nd)$.
- Assume these two steps are each done once for l iterations: $O(lknd)$.

How to Choose K?

Elbow method



Distortion score: computing the sum of squared distances from each point to its assigned center

Limitations of K-Means

- Local minima
 - Can we fix it?
- Cluster shape
 - Can we fix it?
- Susceptibility to outliers
 - Can we fix it?